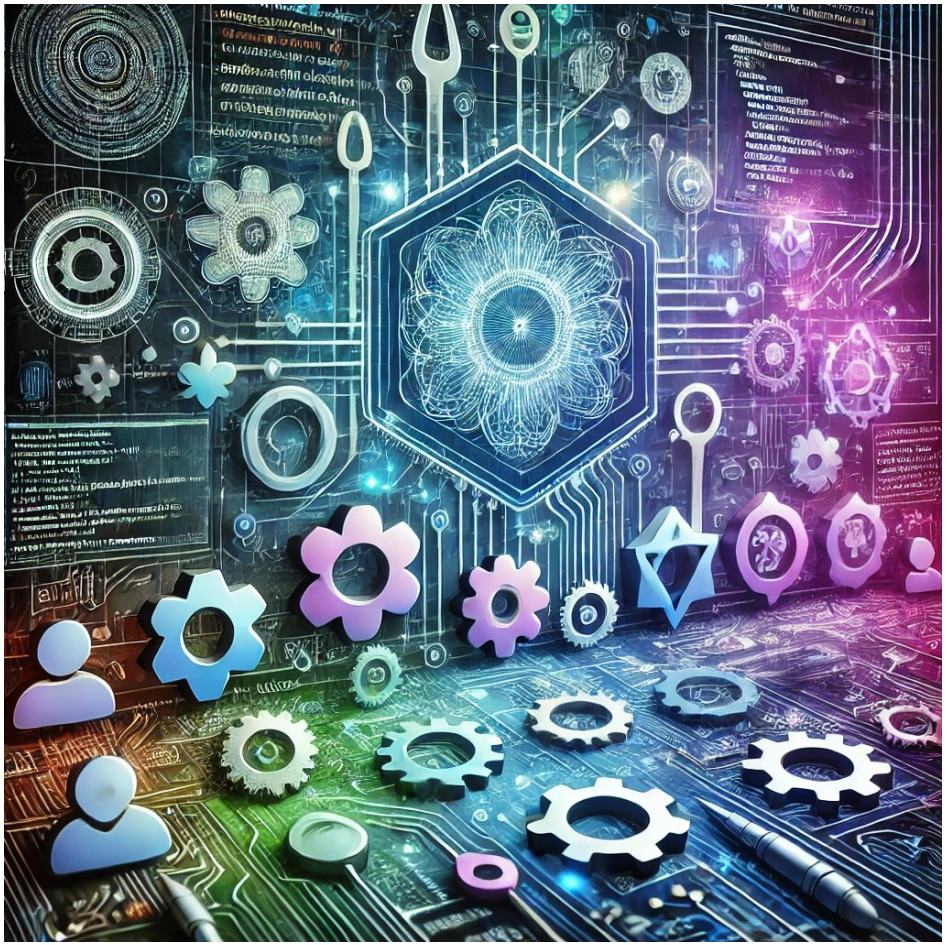
Software Design

FH JOANNEUM
Software Design & Cloud Computing

Schindler/Schalk
Version: 26

CC BY SA

Software Design



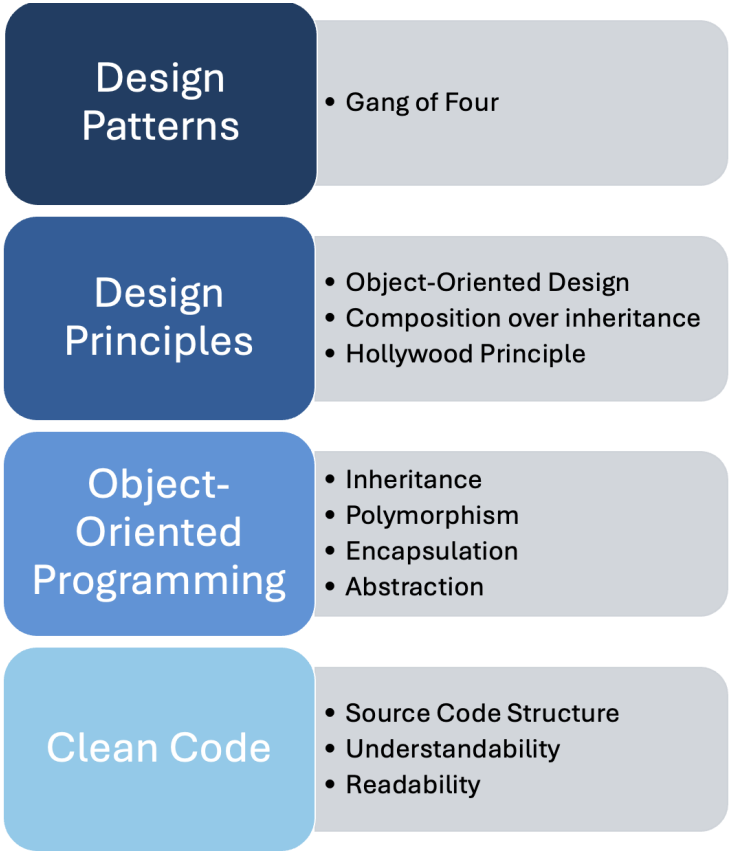
Software Design

What is it all about?

* image contains traces of AI

Software Design

- **Unified Modeling Language (UML)**
  - Class-, Object-, Package-, Sequence Diagram
- **Object-Oriented Design**
  - Principles of Object-Oriented Design
  - (Best of) Design Patterns
  - Creational-, Structural-, Behavioral Patterns
- **Refactoring**



Who am I?

- Name:
  - Ing. **Markus Schalk**, BSc MSc
  - markus.schalk@fh-joanneum.at → no „.edu“
- Education:

  HTL Informatik

  Bachelor/Master Software Engineering
- Experience

  Development & Management:
  - Various roles in R&D engineering department

  Academia:
  - FH Joanneum

Software Design Syllabus

Aims

- Basic knowledge of UML notations.
- Ability to map UML diagrams into Java code and vice versa.
- The competence to correctly apply the presented design patterns in a practical context.

Software Design Syllabus

Organization

- ILV - 4 ECTS
- Details im Syllabus ersichtlich

- **Leistungsüberprüfung 15.01.2027**
  - In Präsenz
  - Schriftlich auf Papier in Einzelarbeit, ohne Hilfsmittel
- **Laborübungen**
  - Implementierungen von Java-Übungen in Mob-Programming-Gruppen

Kriterien:	% oder Punkte	Kriterium muss separat positiv sein?
Leistungsüberprüfung	60	☐ Ja ☒ Nein
Laborübungen	40	☐ Ja ☒ Nein

Code of Conduct

- Your programming environment works
  - In students responsibility
  - Exercises done in the **programming language Java**.
  - You need a working **IntelliJ JetBrains** IDE and min. **Java 17** (JDK LTS) / Miscellaneous: Teams, Editor etc.

- Questions in lectures = yes! ✋💬
- Attendance
- For lab submissions:
  - **Readable and executable** source code
  - **Report** as submission in appealing form
  - **Report** contains names of students in header
  - Appropriate scientific practice (e.g. reference to sources, no plagiarism)

Software Design

Introduction

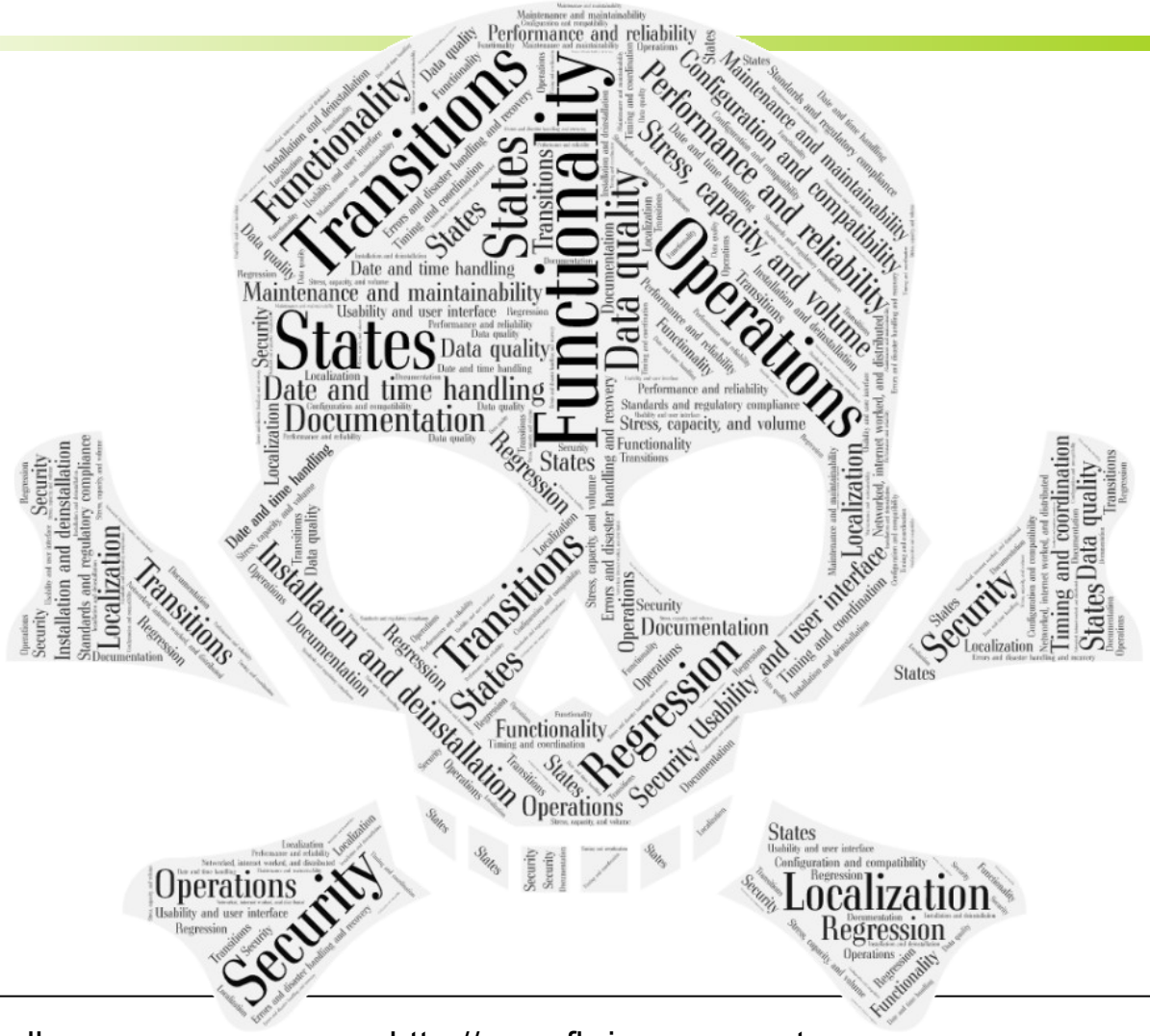


What is Quality?

Quality is **never** an **accident**;
it is always the result of
intelligent effort.

– John Ruskin (1819-1900)

What are Quality Risks?



Why bother about Quality?

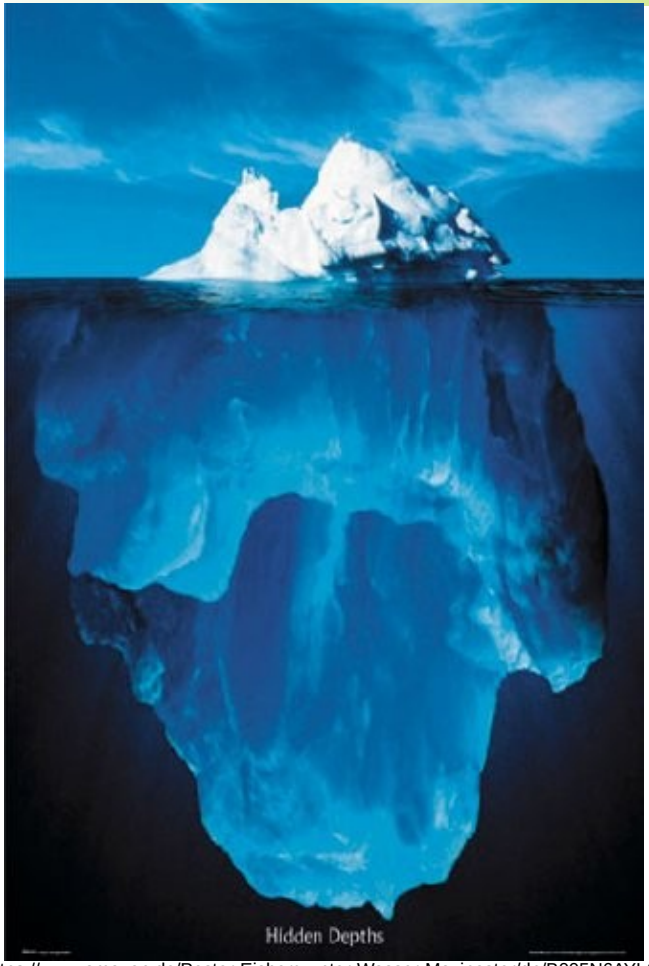
It affect all out lives in various ways…

According the **5th Software Fail Watch** 2017

- identified 606 recorded software failures (English language news articles)
- affected 314 companies
- affecting 3,7 billion people (nearly 50% world population)
- Costs $ 1700 billion in assets

Source: Tricentis - https://www.tricentis.com/resources/software-fail-watch-5th-edition/

Why so much waste?

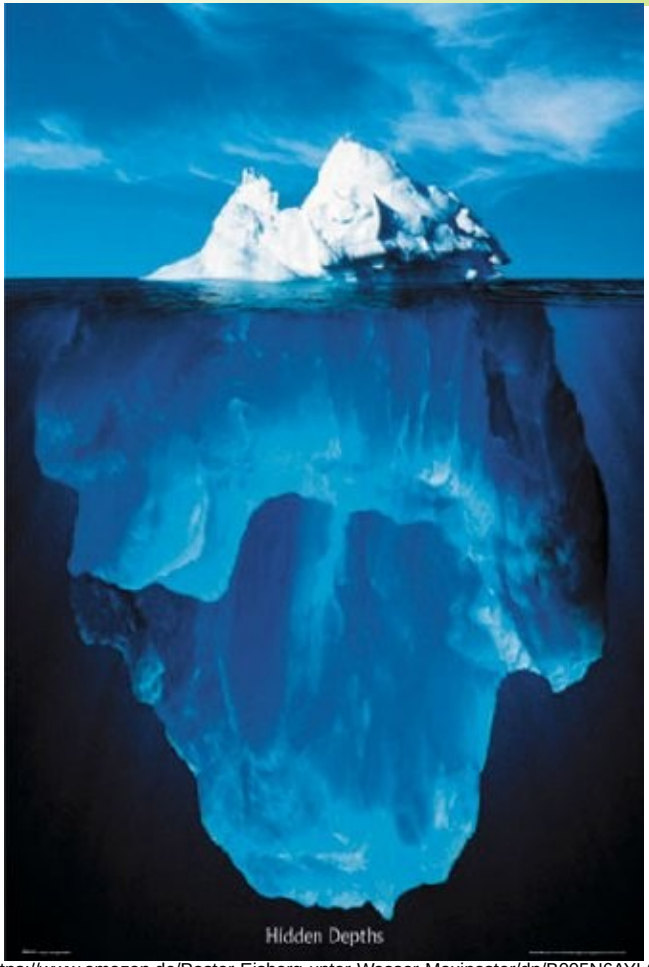


https://www.amazon.de/Poster-Eisberg-unter-Wasser-Maxiposter/dp/B005N6AYL6

- Visible:
  - Customer problem reports
  - Customer service calls
  - Lawsuits/warrantee claims
  - QA & test department costs
  - Service outages

- Invisible:
  - Finding & fixing internal problems/defects
  - Cancelled and troubled projects
  - Unaccounted overtime (crisis mode)
  - Waste and rework
  - Successful cyber attacks
  - Staffing problems (e.g.turnover)
  - Poor teamwork
  - Lack of good planning
  - Dubious project value/ROI
  - Excessive systems costs
  - Lost market opportunities
  - Lack of good practices & quality standards
  - Understanding complex code
  - Technical debt
  - Poor quality data

Why so much waste?



https://www.amazon.de/Poster-Eisberg-unter-Wasser-Maxiposter/dp/B005N6AYL6

- Ratio VISIBLE : INVISIBLE = 1:5
- INVISIBLE quality issues
  are often unnoticed by management
- OUR duty as a SW-devs is to increase the QUALITY through VARIOUS measures – which are in our POWER
- One of the measures, among others like testing is SOFTWARE-DESIGN.

Introduction

Outline

- Introduction
- Modeling Languages
- Questions

- What is Software Design
- Design Principles
- Design Approaches

What is Software Design?

Software design is the process by which an agent
creates a specification of a software artifact,
intended **to accomplish goals**,
using a set of **primitive components**
and **subject to constraints**.

– Paul Ralph and Yair Wand (2009)

What is Software Design?

"Software design usually involves
problem solving and planning a software solution.
This includes both, a low-level component and algorithm design, and a high-level, architecture design."
– Wikipedia

Software design is the **interface between requirements and implementation**.

Requirements: determine **WHAT** the system does
Design: determines **HOW** this is implemented

Software Design Principles

The design…

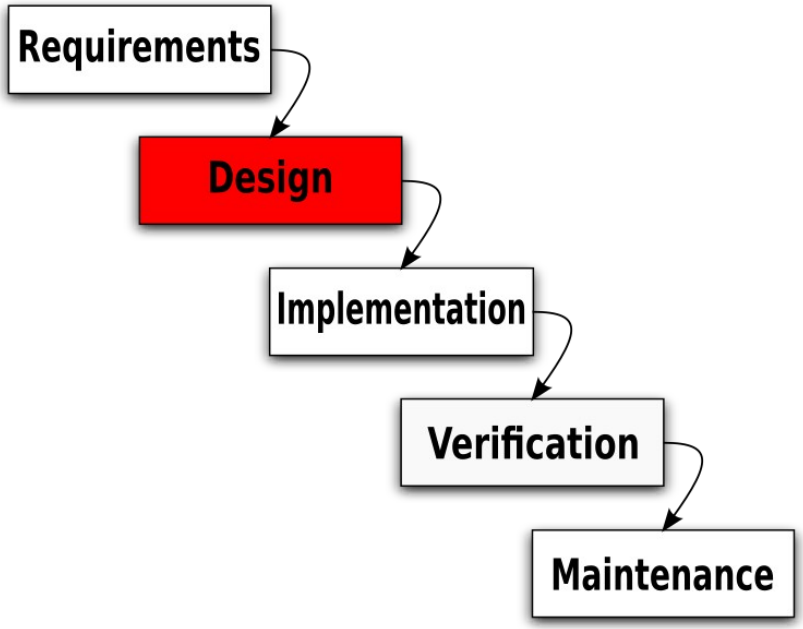
- … process should **not suffer** from **“tunnel vision”**.
- … should **not “reinvent the wheel”**
- … should, whenever possible, mimic the **structure of the problem domain**
- … should **exhibit uniformity** and **integration**.
- … should be structured to **accommodate change**.
- … should be designed to **accommodate unusual circumstances,** if it must terminate, do so in a graceful manner.
- … should be assessed for quality as it is being created
  – **not after** the fact.
- … **should be reviewed** to minimize conceptual errors.

Why (good) Software Design?

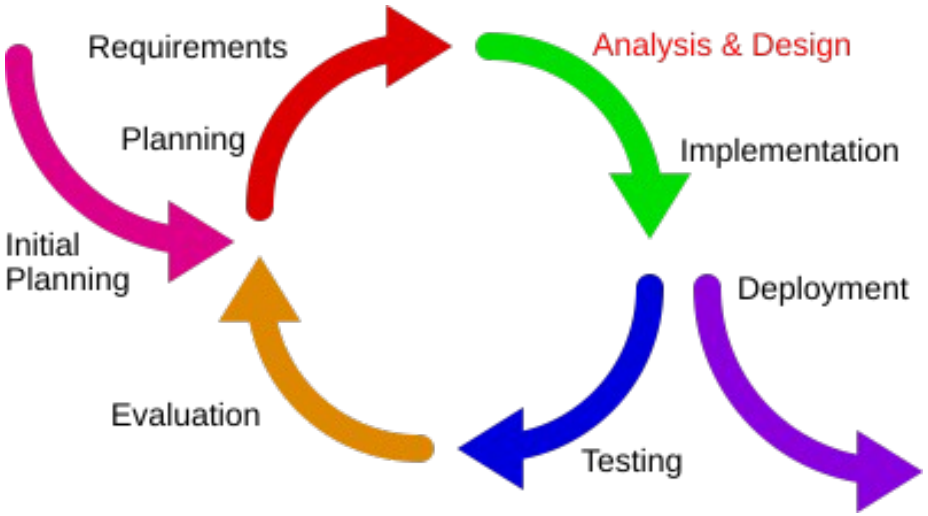
- Software size **is getting larger**
- **Complexity** thus **increases**
- Complexity of problem domain
- Difficulty describing the behavior of discrete systems
- **Lack of adaption to new requirements** leads to restrictions in design, decreasing performance, decreasing reliability
- **Necessary cleanup work** is not carried out leading to missing or incorrect documentation, tests etc.

Design Approaches

- Big Design Up Front (BDUF)



- Iterative & Incremental Design



Design Approaches: **Comparison**

Big Design Up Front (BDUF)

- No feedback
- No design changes
- Use if requirements & the technology is well known
  - implementing protocols
  - implementing algorithms

Iterative & Incremental Design

- Continuous design
- Feedback loop
- Frequent design changes
- Use in complex settings
  - Customer project
  - New technology
  - Market demands

Software Design Approach

Treat design as a wicked, sloppy, heuristic process. Don't settle for the first design that occurs to you. Collaborate. Strive for simplicity. Prototype when you need to. Iterate, iterate and iterate again. You'll be happy with your designs.

– Steve McConnell, Code Complete

Understand and analyze the problem => identify possible solutions

Develop different system models at different levels of abstraction:

- divide and conquer
- iterate, iterate, iterate

Outline revisited

Outline

- Introduction
- **Modeling Languages**
- Questions

- **Flowchart & Structogram**
- Unified Modeling Language (UML)
- Purpose: →Communication

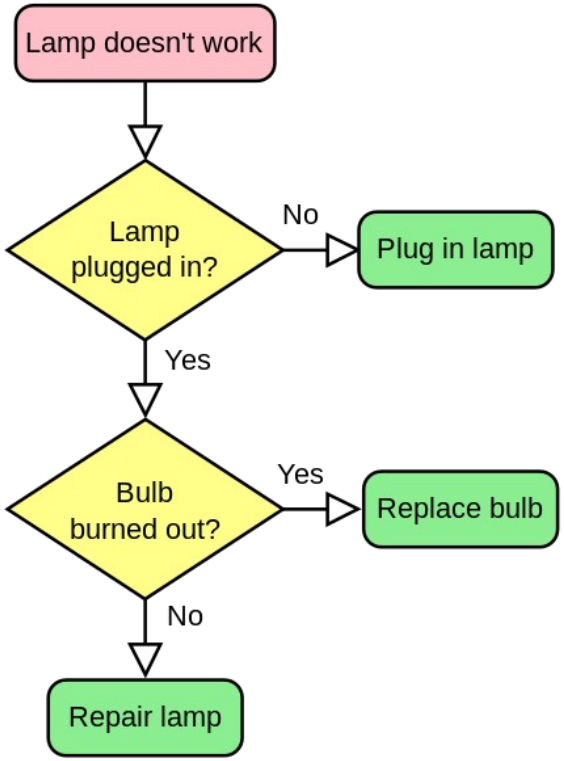
Software Design Principles

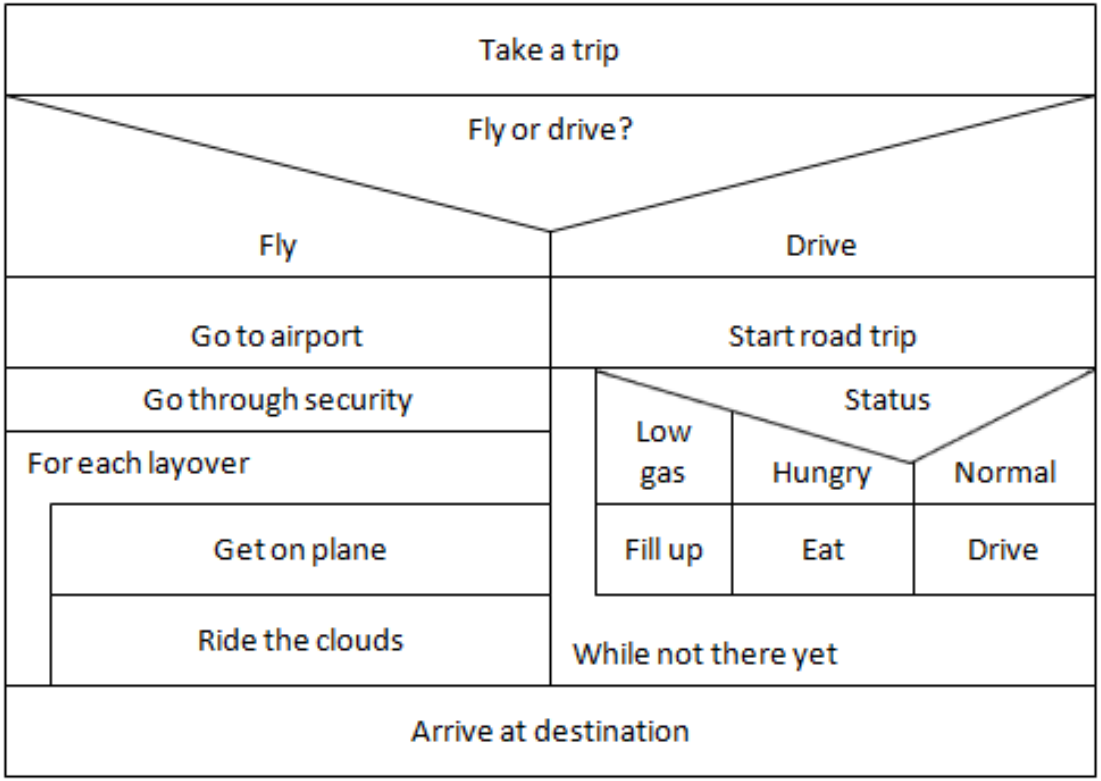
No matter which approach, we need ways **to communicate our design**. Different modeling languages have been created for that purpose. We will have a look at:

- Structogram [1] (DIN 66261)
- Flowchart (DIN 66001)
- **Unified Modeling Language (UML)**

[1] Nassi-Schneidermann diagram

Flowchart & Structogram





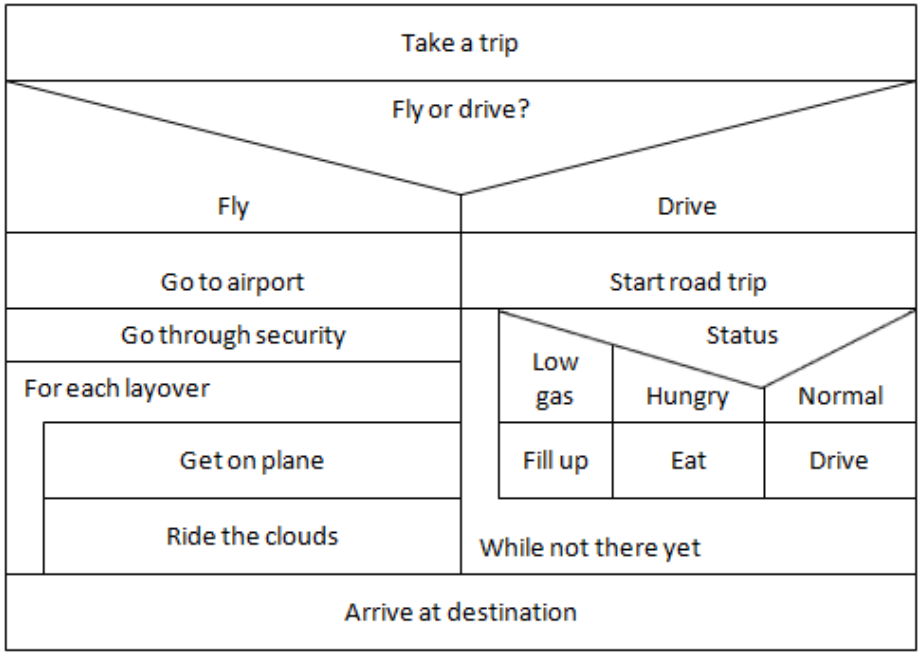
Both diagrams allow the description of **control flows**. They are (almost) interchangeable. Flowcharts seem to be used more widely and more often.

Structogram

Used a graphical design representation for structured programming.

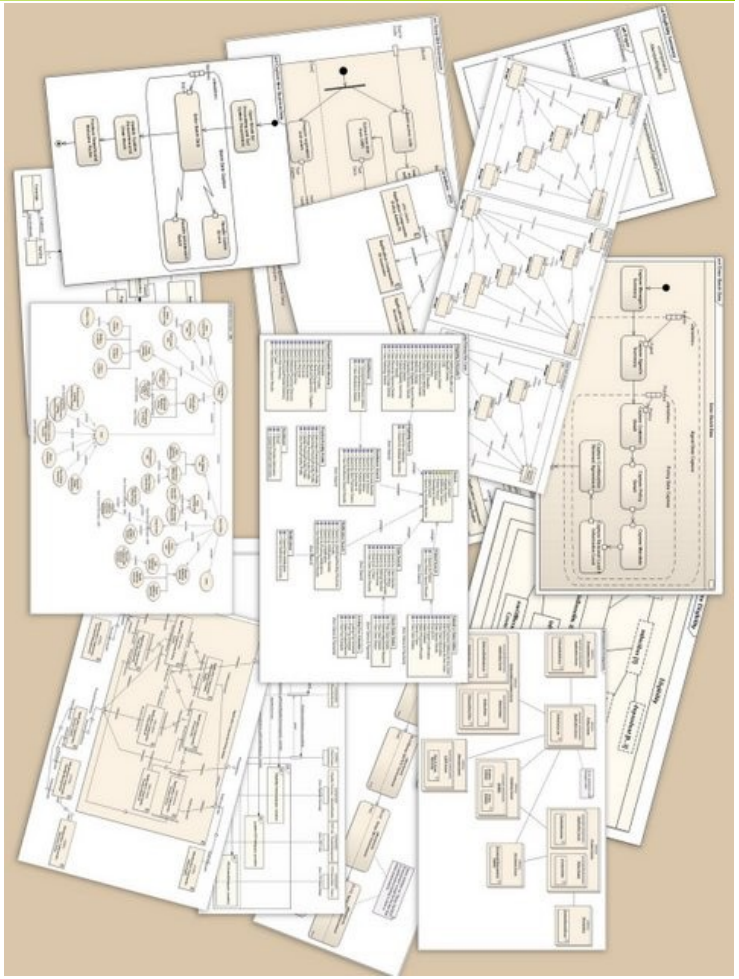
Following a top-down design, the problem at hand is reduced into smaller and smaller subproblems, until only simple statements and control flow constructs remain.

A structogram reflects this top-down decomposition in a straightforward way, using nested boxes to represent subproblems.



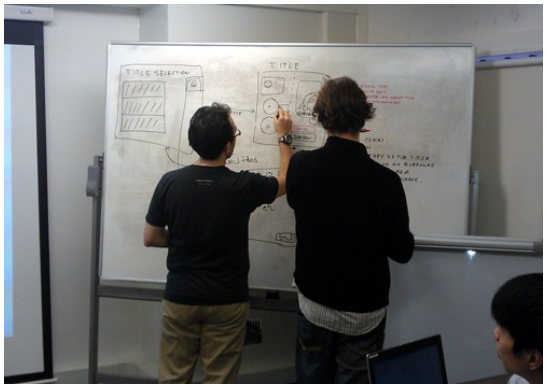
Unified Modeling Language

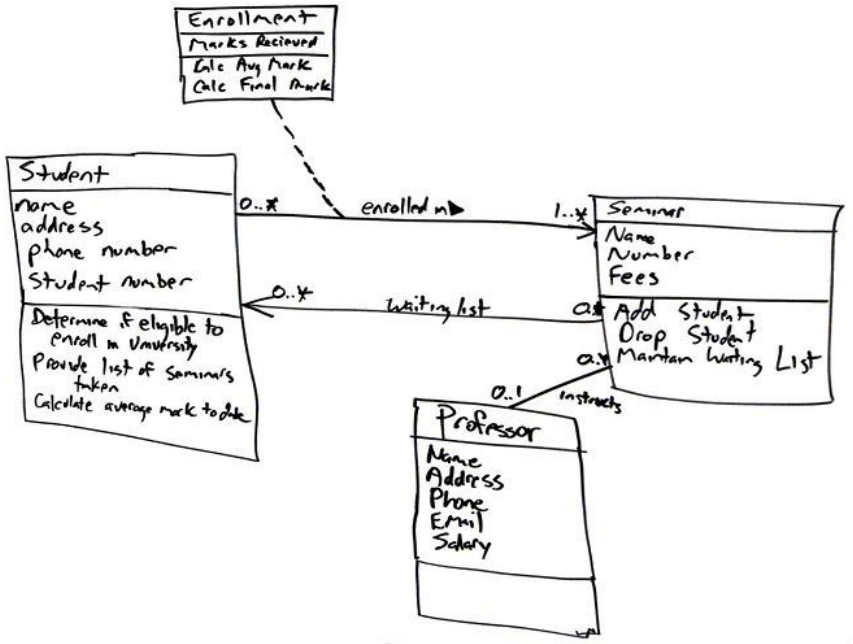
- General modeling language
- Software description
  - structural
  - behavioral
- Graphic notation
- Multiple diagram types
- Widely used
- Allows for extension



Purpose Communication

Remember: **goal** is **communication** not nice diagrams!





Questions?



FAQs

- What is SW-Design? What is involved?
- Name some (at least three) SW-Design principles.
- When does SW-Design occur in the BDUF approach? And what does that imply?
- What modeling languages do you know?
- What does UML stand for? What aspects of Software does it describe?
- What is the main purpose of modeling languages?